

13/01/2026

Compléments SAÉ S3

Qualité, Documentation, Gestion du changement

Projet : Application web de vote football Europe (Top 5 ligues)

Technos : PHP, JavaScript, CSS, MySQL/MariaDB, Apache, HTTPS/SSL, GitHub

Équipe : Damien Rinaldis & Ethan Stasse

I. La qualité

A. Les différents aspects de la qualité

1) Réponse technique (qualité “fonctionnelle” côté système)

L'application est conçue pour fonctionner de manière fiable dans un environnement web classique, avec un serveur Apache, une base MySQL/MariaDB et un échange sécurisé via HTTPS. Les fonctions principales, comme l'affichage des ligues et des équipes, le vote, et le calcul ainsi que l'affichage des résultats, reposent sur des traitements simples et maîtrisés. Cela inclut des requêtes SQL paramétrées, des pages PHP séparées par responsabilité, et des scripts JavaScript limités à l'interactivité.

La qualité technique prend également en compte la sécurité. Cela comprend la validation des entrées, des protections contre les attaques web courantes, une configuration serveur cohérente avec HTTPS et des entêtes de sécurité, ainsi qu'une gestion des erreurs qui évite les fuites d'informations sensibles.

2) Réponse à l'utilisation (qualité “expérience utilisateur”)

L'application est conçue pour une utilisation rapide, avec un parcours utilisateur court : sélection d'une ligue, choix d'une équipe, vote, puis accès au classement et aux résultats. L'ergonomie se concentre sur la simplicité. Elle offre des libellés clairs, une interface responsive et un retour utilisateur, comme la confirmation du vote et des messages d'erreur compréhensibles.

La qualité de l'utilisation se mesure aussi par la prévention des erreurs. Cela inclut le contrôle des champs, des messages explicites et une cohérence globale entre les écrans.

13/01/2026

3) Réponse globale à un problème (qualité “produit”)

Le projet répond à un besoin clair. Il permet aux utilisateurs de comparer des équipes des 5 grandes ligues européennes grâce à un système de vote. Ils peuvent aussi voir un résultat représentatif. La solution respecte le cahier des charges en offrant un fonctionnement facile à comprendre, une base de données bien organisée, et une structure simple à déployer sur un serveur local ou distant.

B. Qualité logicielle selon ISO/IEC 25010 (adaptée au projet)

1) Adéquation fonctionnelle

Les fonctionnalités attendues sont couvertes. On peut consulter les ligues et les équipes. Il est possible de voter. Les votes sont stockés. Les résultats sont calculés et affichés. Les règles métiers, comme la limitation du vote, l'intégrité des données et la cohérence des ligues et des équipes, sont gérées par le serveur.

2) Performance / efficience

Les pages utilisent des requêtes spécifiques pour améliorer leur efficacité. Cela signifie que l'on peut indexer les clés étrangères et les champs qui sont souvent utilisés pour filtrer les résultats. Les traitements lourds sont évités autant que possible, et on utilise la pagination lorsque cela est nécessaire. L'objectif principal est de garantir une navigation fluide, même lorsqu'il y a un grand volume de votes.

3) Compatibilité

L'application fonctionne correctement dans n'importe quel navigateur moderne, et elle ne nécessite pas d'abonnement payant. Les interactions avec le client sont conçues pour être simples, ce qui aide à réduire les risques d'incompatibilités. De plus, l'application est hébergée sur une plateforme standard composée d'Apache, PHP et MariaDB, ce qui facilite sa reproduction et sa mise en œuvre.

4) Utilisabilité

Le parcours utilisateur est direct : peu d'étapes, interface claire, prévention des erreurs. Les libellés et les messages d'état (succès/échec) sont prévus. Une documentation utilisateur légère est possible (guide ou tutoriel).

13/01/2026

5) Fiabilité

L'intégrité des données est assurée grâce à des contraintes SQL telles que les clés primaires et les clés étrangères. Des transactions peuvent également être utilisées si nécessaire lors de l'écriture de vote. La gestion des erreurs est également gérée de manière appropriée. L'objectif principal est d'éviter les doublons, les incohérences et les états qui sont impossibles. L'intégrité des données est donc très importante pour garantir la cohérence et la fiabilité des données.

6) Sécurité

La sécurité est prise en compte de différentes manières. Elle est traitée au niveau du transport, ce qui signifie qu'elle utilise HTTPS. La sécurité est également traitée au niveau du code, avec des validations et des requêtes préparées. Les sessions sont sécurisées, ainsi que les protections anti-CSRF. Il y a des limitations pour lutter contre le spam et les abus. Les entêtes HTTP sont également utilisés pour la sécurité, ainsi que le contrôle d'accès. Une section spécifique sur la sécurité est détaillée plus bas.

7) Maintenabilité

Le code est organisé par responsabilités (affichage, accès aux données, logique métier, sécurité). La présence de bonnes pratiques (conventions, fonctions réutilisables, commentaires, structure MVC légère si appliquée) facilite les évolutions.

8) Portabilité

Déploiement possible sur différentes machines/serveurs tant qu'un stack Apache/PHP/MySQL est disponible. Les paramètres sensibles (accès DB, secrets) sont externalisés (fichier de configuration non versionné).

C. Démarche qualité mise en œuvre

1) Adéquation fonctionnelle

Les fonctionnalités attendues sont couvertes. Nous pouvons consulter les ligues et les équipes. Il est possible de voter et les votes sont stockés. Les résultats sont calculés et affichés. Les règles métiers, comme la limitation du vote, l'intégrité des données et la cohérence des ligues et des équipes, sont prévues côté serveur.

2) Performance / efficience

Les pages utilisent des requêtes ciblées. Cela signifie que l'on peut indexer sur les clés étrangères et les champs qui sont fréquemment filtrés. On évite les traitements lourds dans la mesure du possible. Si cela est nécessaire, on utilise la pagination pour améliorer les performances. L'objectif principal est de garantir une navigation fluide, même lorsque le volume de votes est élevé.

3) Compatibilité

L'application fonctionne correctement dans n'importe quel navigateur moderne sans nécessiter d'abonnement payant. Les interactions avec le client sont conçues pour être simples, ce qui aide à minimiser les problèmes d'incompatibilité. De plus, l'application est conçue pour fonctionner avec un hébergement standard basé sur Apache, PHP et MariaDB, ce qui la rend facile à reproduire et à mettre en œuvre.

4) Utilisabilité

Le parcours utilisateur est direct : peu d'étapes, interface claire, prévention des erreurs. Les libellés et les messages d'état (succès/échec) sont prévus. Une documentation utilisateur légère est possible (guide ou tutoriel).

5) *Fiabilité*

L'intégrité des données est garantie grâce à des contraintes SQL, comme les clés primaires et les clés étrangères. Les transactions sont également utilisées si nécessaire lors de l'enregistrement d'un vote. De plus, les erreurs sont gérées de manière appropriée. L'objectif principal est d'éviter les doublons, les incohérences et les situations impossibles dans les données.

6) *Sécurité*

La sécurité est considérée comme très importante et elle est assurée à plusieurs niveaux. Elle commence par le transport, qui utilise HTTPS pour une transmission sécurisée. Ensuite, elle est renforcée au niveau du code, qui implémente la validation et utilise des requêtes préparées pour minimiser les risques. Les sessions sont également sécurisées, et des protections anti-CSRF sont mises en place pour empêcher les attaques de forgery de requêtes. De plus, des limitations sont établies pour lutter contre le spam et les abus, et les entêtes HTTP sont configurés pour améliorer la sécurité. Enfin, un contrôle d'accès strict est mis en place pour garantir que seules les personnes autorisées puissent accéder aux données sensibles. Une section dédiée à la sécurité est détaillée plus bas pour fournir plus d'informations sur ces mesures.

7) *Maintenabilité*

Le code est organisé par responsabilités (affichage, accès aux données, logique métier, sécurité). La présence de bonnes pratiques (conventions, fonctions réutilisables, commentaires, structure MVC légère si appliquée) facilite le Versioning et traçabilité (GitHub)

Le projet utilise GitHub pour le versionnement. Cela signifie que l'historique des modifications est conservé. Il est donc possible de revenir en arrière si besoin. Le suivi des corrections est également assuré. Et si cela est nécessaire, des branches peuvent être créées pour les nouvelles fonctionnalités ou les corrections, comme par exemple feature ou fix. Cette façon de procéder réduit les risques de perdre des données et améliore la qualité du projet grâce à un suivi clair et précis.

13/01/2026

Organisation du travail (binôme)

Nous avons travaillé ensemble sur l'ensemble du projet, Damien s'étant davantage concentré sur le front-end, tout en concevant conjointement la base de données, les pages PHP, la sécurité du site et les tests afin d'assurer le bon fonctionnement global de l'application.

Les choix techniques effectués par nous ont bien été documentés et cohérents.

13/01/2026

Tests

Les tests manuels structurés comprennent des tests de vote et des tests de validation. Ces tests de validation vérifient les champs vides et les formats invalides. Il faut également effectuer des tests d'accès, qui concernent la session, ainsi que des tests de sécurité de base. Ces tests de sécurité de base incluent la protection contre les injections et les attaques XSS. Les cas les plus critiques, tels que le vote, l'enregistrement dans la base de données et l'affichage du classement, sont traités en priorité.

Qualité de code / principes SOLID (adaptation au PHP web)

Même sur un projet PHP, on applique l'esprit SOLID pour garder un code propre :

- S (Single Responsibility) : une classe/fonction = une responsabilité (ex : VoteRepository pour DB, VoteService pour règles, Controller pour HTTP).
- O (Open/Closed) : ajouter une ligue ou une règle sans casser l'existant (ex : configuration des ligues, pas de "switch" partout).
- L (Liskov) : les interfaces sont respectées, comme par exemple RepositoryInterface.
- I L'interface Segregation, également appelée Interface Segregation Principle, est une pratique de conception qui consiste à diviser les grandes interfaces en de petites interfaces plus utiles. Cela se traduit par la création de petites interfaces qui répondent à des besoins spécifiques, comme par exemple le VoteRepository ou le TeamRepository. Le but est de rendre les interfaces plus faciles à utiliser et à comprendre, en leur donnant des responsabilités plus précises.
- D (Dependency Inversion) : le contrôleur dépend d'abstractions (interfaces), pas d'une classe DB directement.

Concrètement : séparation "accès DB" / "logique vote" / "affichage", et un fichier config pour l'environnement.

Sécurité (mesures gratuites, faisables en code/config)

1) Injection SQL (MySQL/MariaDB) — priorité absolue

Lorsque vous travaillez avec une base de données, il est essentiel d'utiliser des requêtes préparées via PDO. Vous devez utiliser `bindValue` ou `execute` pour les requêtes. Il est important de ne jamais incorporer directement les données des utilisateurs dans les requêtes SQL.

Les données que vous recevez doivent être vérifiées côté serveur. Par exemple, vous devez vérifier que les identifiants de ligue ou d'équipe sont bien des nombres.

Le compte que vous utilisez pour vous connecter à la base de données doit avoir les privilèges minimum. Il ne doit avoir accès qu'aux opérations nécessaires comme la lecture, l'insertion et la mise à jour des données. Il ne doit pas avoir des droits étendus comme `GRANT` ou `FILE`.

2) XSS (injection JavaScript via champs ou URL)

Toute donnée affichée dans une page HTML doit être systématiquement échappée à l'aide de fonctions comme `htmlspecialchars`. Cette fonction prend pour paramètres la valeur à échapper, le type de guillemets à prendre en compte et le jeu de caractères utilisé. Dans ce cas, nous utilisons `ENT_QUOTES` pour les guillemets et `'UTF-8'` pour le jeu de caractères. Il ne faut jamais injecter directement du contenu provenant de l'utilisateur dans le HTML ou le JavaScript sans un encodage préalable. Lorsque cela est possible, l'utilisation d'une Content Security Policy restrictive permet de limiter l'exécution de scripts non autorisés.

3) CSRF (soumission de formulaire à l'insu de l'utilisateur)

Chaque formulaire qui utilise la méthode POST doit avoir un token CSRF. Ce token est généré et stocké dans une session. Lorsque le serveur reçoit la requête, il doit vérifier ce token pour s'assurer que l'action vient vraiment de l'utilisateur qui a le droit de la faire.

4) Authentification et gestion des sessions (si administration ou limitation de vote)

Les cookies de session doivent être configurés avec les options de sécurité appropriées. Cela signifie que le cookie_secure doit être activé lorsqu'on utilise HTTPS. De plus, HttpOnly doit être activé et SameSite doit être réglé sur Lax ou Strict, selon le contexte dans lequel cela est utilisé. Lorsqu'une authentification est réussie, l'identifiant de session doit être régénéré. Il est également important de mettre en place une expiration automatique de la session et une déconnexion propre pour limiter les risques d'usurpation.

5) Validation des entrées (client et serveur, serveur obligatoire)

Côté serveur, il est nécessaire de vérifier les identifiants numériques autorisés. Il faut s'assurer que seuls les identifiants numériques autorisés sont acceptés. Les champs attendus doivent être présents. Les tailles maximales doivent être respectées. Les caractères autorisés doivent correspondre au format prévu. Toute donnée qui ne respecte pas ces règles doit être rejetée par défaut. Cela fonctionne selon le principe du fail closed. Les données qui ne sont pas autorisées doivent être rejetées par défaut. Cela signifie que le serveur doit rejeter toute donnée qui ne correspond pas aux règles établies.

6) Gestion des erreurs (ne pas aider un attaquant)

Les messages d'erreur SQL ou techniques ne doivent pas être affichés en clair dans un environnement de production.

Les erreurs doivent être enregistrées dans des fichiers de logs sur le serveur.

L'utilisateur doit uniquement voir un message générique qui ne révèle aucune information sensible.

7) Mots de passe (si des comptes utilisateurs existent)

Les mots de passe doivent être stockés uniquement sous forme de hash à l'aide de la fonction `password_hash()` avec un algorithme adapté comme `bcrypt` ou `argon2`. Lors de l'authentification, la vérification doit se faire avec `password_verify()`. Une politique minimale de sécurité doit être appliquée, incluant une longueur suffisante et une limitation du nombre de tentatives.

8) Limitation des abus et anti-spam de vote

Un mécanisme de limitation de fréquence peut être mis en place côté serveur. Cela signifie que le serveur peut être configuré pour permettre un seul vote par intervalle défini. Pour mettre en œuvre cela, il est possible de stocker un identifiant de vote basé sur la session et un timestamp. Alternativement, si une authentification est utilisée, des règles peuvent être appliquées par compte utilisateur. Même sans solution payante, une limitation basée sur l'adresse IP et le temps peut déjà être suffisante pour réduire les abus. Dans ce cas, un captcha peut rester une option supplémentaire pour renforcer la sécurité.

9) HTTPS et SSL (sécurisation du transport)

Le protocole HTTPS doit être forcé via une redirection permanente (301). Les anciennes versions de TLS doivent être désactivées lorsque la configuration du serveur le permet. L'en-tête HSTS (Strict-Transport-Security) peut être activé afin d'imposer l'utilisation du HTTPS pour toutes les connexions futures.

10) Entêtes HTTP de sécurité (solutions gratuites)

L'ajout d'entêtes HTTP comme X-Content-Type-Options, X-Frame-Options, Referrer-Policy et Permissions-Policy permet de renforcer la sécurité globale de l'application web. Il est possible d'appliquer une Content Security Policy de manière progressive en fonction des besoins et des contraintes spécifiques du projet en question.

11) Téléversement de fichiers (si cette fonctionnalité est ajoutée ultérieurement)

Par défaut, il est préférable d'interdire le téléversement de fichiers. Si le téléversement de fichiers devient nécessaire, il faudra alors limiter les extensions de fichiers autorisées. Il faudra également vérifier le type de fichier, qui est appelé type MIME. Les fichiers téléversés doivent être stockés en dehors du répertoire web. Enfin, il faudra attribuer des noms de fichiers aléatoires aux fichiers téléversés.

12) Configuration du serveur Apache

Il est recommandé de désactiver le listing des répertoires afin d'éviter l'exposition de fichiers internes. Les fichiers sensibles tels que les fichiers de configuration ou les variables d'environnement doivent être protégés par des règles d'accès strictes. Les permissions sur les fichiers doivent être minimales et les configurations de développement et de production clairement séparées.

II. La documentation

Documentation organisationnelle

La documentation organisationnelle présente le projet, ses objectifs et son périmètre, à savoir une application web de vote sur les équipes des cinq grandes ligues européennes. Elle précise également la répartition des tâches entre Damien Rinaldis et Ethan Stasse. Le suivi du projet est assuré à l'aide de GitHub, permettant la gestion des versions du code et la traçabilité des modifications.

Documentation technique

La documentation technique décrit l'architecture générale de l'application. Les pages principales constituent l'interface utilisateur, tandis que les contrôleurs gèrent la logique applicative et les échanges avec la base de données. Celle-ci permet de stocker les informations relatives aux ligues, aux équipes et aux votes. Des scripts JavaScript sont utilisés pour l'interactivité et des feuilles de style CSS pour la présentation. Les choix techniques incluent l'utilisation de PDO avec requêtes préparées, des mécanismes de sécurité comme les tokens CSRF et les sessions sécurisées, ainsi que le déploiement sur un serveur Apache avec configuration d'hôte virtuel, SSL et import de la base de données.

Documentation utilisateur

La documentation utilisateur prévoit un guide simple expliquant comment choisir une ligue, sélectionner une équipe, voter et consulter les résultats. Une FAQ permet de répondre aux questions courantes, comme l'impossibilité de voter plusieurs fois ou un vote non pris en compte. En cas de diffusion réelle, un court tutoriel vidéo pourrait également être proposé.

III. Gestion du changement (projection diffusion réelle)

Si l'application était déployée pour de vrais utilisateurs, la gestion du changement viserait éviter les ruptures et accompagner l'usage.

- Communication des changements
- Page “Nouveautés / Changelog” (ex : ajout d'une ligue, correction vote).
- Message d'information lors de la connexion ou en haut du site.
- Déploiement et mises à jour
- Déploiement via Git (tag de version), sauvegarde DB avant mise à jour.
- Mise à jour progressive : d'abord test en environnement de préproduction/local, puis production.
- Support et retours
- Formulaire de retour/bug report.

13/01/2026

Nous allons analyser les retours pour déterminer les corrections à effectuer en priorité, comme les votes, les problèmes de lenteur ou les erreurs d'affichage.

- Formation / accompagnement
- Mise à jour du guide utilisateur après chaque évolution.
- FAQ enrichie selon les retours.

Ce travail complémentaire montre que le projet ne se limite pas au développement : la qualité (technique, usage, produit), la documentation, et l'anticipation des changements sont pris en compte. L'application s'appuie sur un socle technique standard (PHP/Apache/MySQL), une organisation de travail en binôme et GitHub, ainsi que des mesures de sécurité réalistes et gratuites, adaptées à un contexte web.